



PROJECT REPORT

Topic:

Serverless Application using AWS Lambda (Free Tier)

Submitted By

- ❖ Eman Waheed (027)
- ❖ Fatima Sabir (034)
- ❖ Laiba Rani (053)
- ❖ Sania Shafique (078)
- ❖ Sania Arshad (098)

Submitted To

- ❖ Mam Madiha Wahid

Semester:

- ❖ 6(A)

Parallel & Distributed Computing

Contents

Abstract.....	3
1. Introduction.....	3
2. Problem Statement.....	3
3. Objectives	4
4. Serverless Computing	4
Features.....	4
5. Tools and Technologies Used.....	4
6. System Architecture.....	4
Architecture Flow	5
Working Flow	5
7. Implementation Methodology.....	5
Step 1 AWS Account Setup.....	5
Step 2: Lambda Function Creation	6
Step 3: Lambda Code.....	7
Step 4: Lambda Testing	8
Step 5:API Gateway Integration	8
Step 6:Deployment.....	9
Step 7:Browser Testing.....	10
8. Algorithm.....	10
9. Results.....	11
10. Advantages.....	11
11. Challenges Faced	11
12. Applications	11
13. Limitations	12
14. Future Enhancements.....	12
15. Conclusion	12
16. References.....	12

Abstract

This project presents the implementation of a serverless application using AWS Lambda and API Gateway under AWS Free Tier. The purpose of the project is to understand cloud-based serverless computing where code runs without managing servers.

A Lambda function was created using Python and integrated with API Gateway to provide a REST endpoint. When a user sends a request through the browser, the request reaches API Gateway, triggers the Lambda function, and returns a dynamic response.

This project demonstrates scalable, event-driven and low-cost computing using cloud services.

1. Introduction

Cloud computing has transformed application deployment by providing on-demand computing resources.

Traditionally, developers had to:

- Configure servers
- Manage scaling
- Handle maintenance
- Apply security patches

These tasks increase operational complexity.

Serverless computing solves this issue by allowing developers to focus only on code while cloud providers manage infrastructure.

AWS Lambda is Amazon's serverless computing service that runs code automatically in response to events.

This project uses:

- AWS Lambda
- API Gateway
- Python Runtime
- AWS Free Tier

to build a simple serverless web application.

2. Problem Statement

Traditional application deployment requires managing servers, which increases:

- Cost

Parallel & Distributed Computing

- Maintenance
- Complexity

The goal of this project is to build a serverless application that removes server management and provides scalable cloud execution.

3. Objectives

Objectives of this project are:

- Understand serverless computing concepts
- Learn AWS Lambda deployment
- Integrate API Gateway with Lambda
- Create and test API endpoints
- Explore cloud-based event-driven architecture

4. Serverless Computing

Serverless computing is a cloud model where developers deploy code without managing servers.

Servers still exist, but are handled by the cloud provider.

Features

- Auto scaling
- Pay-per-use
- High availability
- Reduced maintenance
- Event driven execution

5. Tools and Technologies Used

Technology	Purpose
AWS Lambda	Run serverless code
API Gateway	Create HTTP endpoint
Python	Function programming
AWS Free Tier	Free cloud resources
Browser	API testing

6. System Architecture

Architecture Flow

Client Browser
↓
API Gateway
↓
AWS Lambda
↓
Process Request
↓
Return Response

Working Flow

User sends request:

/hello?name=Sania

API Gateway receives request and invokes Lambda.

Lambda processes request and returns:

Hello Sania, Serverless working!

7. Implementation Methodology

Step 1: AWS Account Setup

Created AWS Free Tier account.

Parallel & Distributed Computing

Choose your account plan



Free (6 months)
Learn, experiment, and build prototypes

- ✓ Receive up to \$200 in credits
- ✓ Includes free usage of select services
- ✗ Workloads scale beyond credit thresholds
- ✗ Access to all AWS services and features

① After the 6 month free period or when all credits are used, you can choose to upgrade to a paid plan. Otherwise, your account closes automatically.

[Choose free plan](#)



Paid
Develop production-ready workloads

- ✓ Receive up to \$200 in credits
- ✓ Includes free usage of select services
- ✓ Workloads scale beyond credit thresholds
- ✓ Access to all AWS services and features

① After all of your credits are used, you are charged using pay-as-you-go pricing.

[Choose paid plan](#)

[View additional details](#)

Activate Windows
Go to Settings to activate Windows.

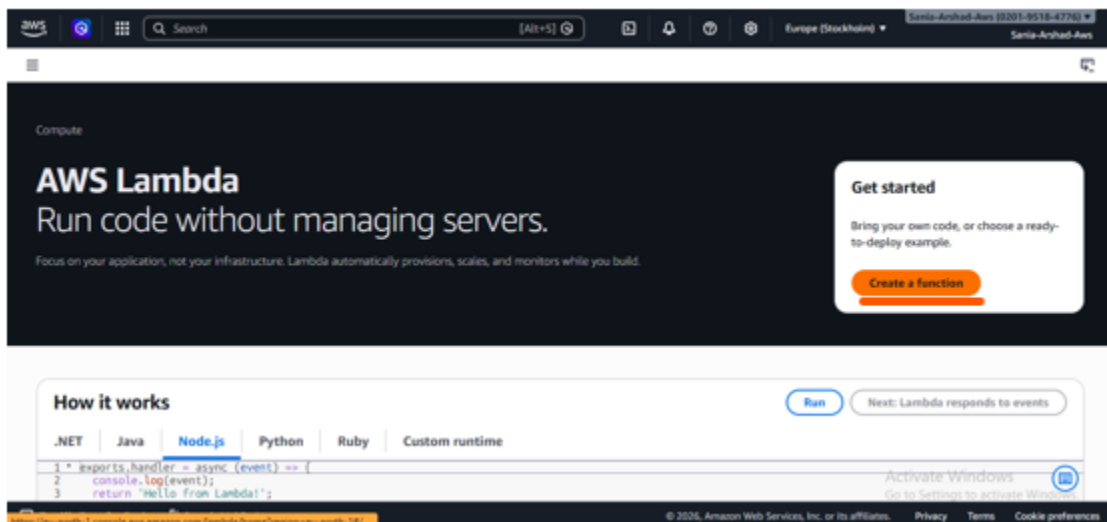
Step 2: Lambda Function Creation

Created Lambda function:

pdCLambda

Runtime selected:

Python 3.x



The screenshot shows the AWS Lambda console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and a user profile. The main content area has a dark header with the text 'Compute' and 'AWS Lambda Run code without managing servers.' Below this, there's a 'Get started' section with a 'Create a function' button. A 'How it works' section is visible at the bottom, showing a code snippet for a Python handler function. The code is as follows:

```
1 * exports.handler = async (event) => {
2   console.log(event);
3   return 'Hello from Lambda!';
}
```

Parallel & Distributed Computing

Create function Info

Choose one of the following options to create your function.

- ☒ **Author from scratch**
Start with a simple Hello World example.
- ☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.
pdclambda
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.14

Durable execution - new Info
Enable durable execution to simplify building resilient multi-step applications that checkpoint progress and resume after interruptions. Supports Python and Node.js runtimes.
[View pricing](#)
☐ Enable

Info **Tutorials**

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

[Start tutorial](#)

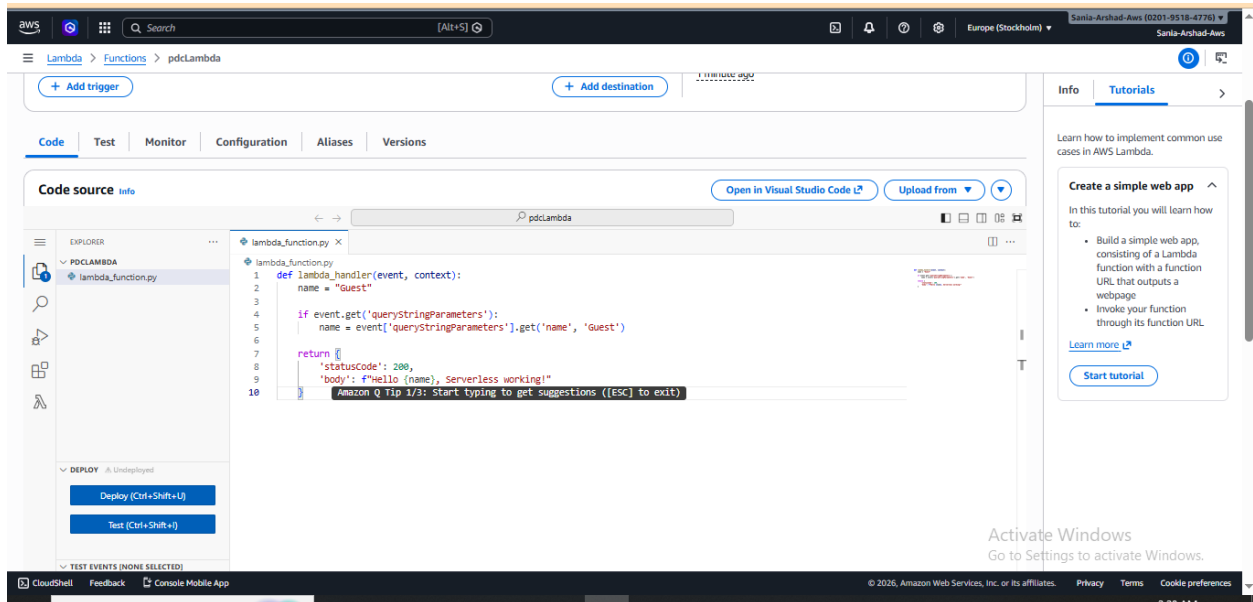
Activate Windows
Go to Settings to activate Windows.

Step 3: Lambda Code

Implemented:

```
def lambda_handler(event, context):  
    name="Guest"  
    if 'queryStringParameters' in event:  
        name=event['queryStringParameters'].get('name','Guest')  
  
    return {  
        'statusCode':200,  
        'body':f"Hello {name}, Serverless working!"  
    }
```

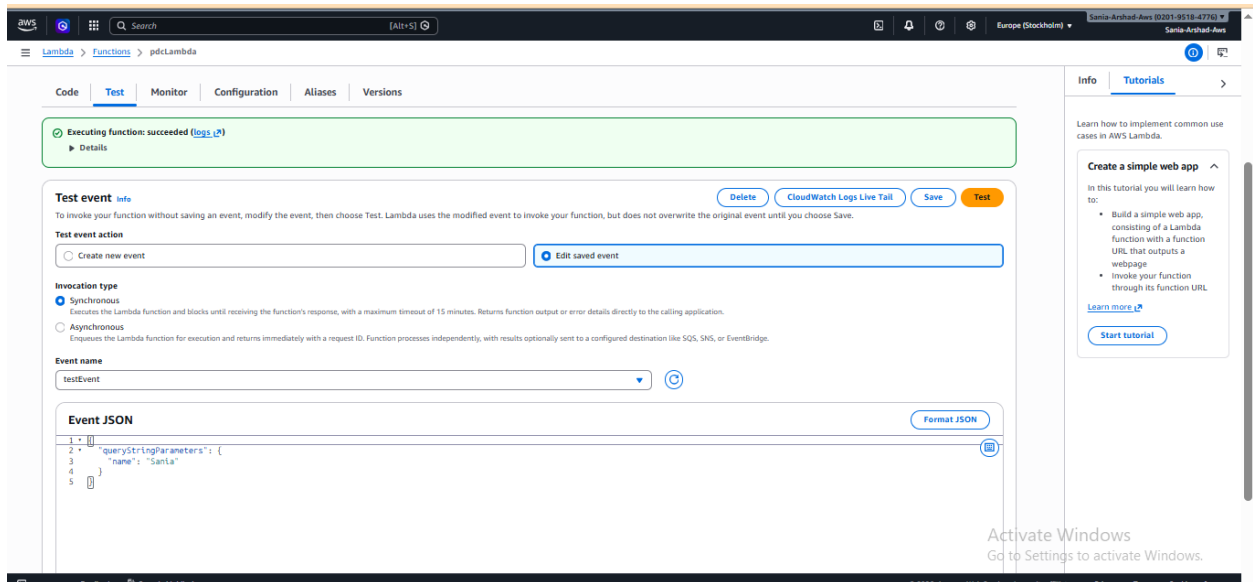
Parallel & Distributed Computing



Step 4: Lambda Testing

Created test event:

```
{
  "queryStringParameters": {
    "name": "Sania"
  }
}
```



Step 5: API Gateway Integration

Parallel & Distributed Computing

Created HTTP API.

Configured route:

GET /hello

Integrated route with Lambda function.

The screenshot shows the AWS API Gateway console interface. On the left, a navigation pane lists the steps: Step 1: Configure API, Step 2 - optional: Configure routes (highlighted), Step 3 - optional: Define stages, and Step 4: Review and create. The main content area is titled 'Configure routes - optional' and contains a section 'Configure routes' with an 'info' icon. Below this, there is a table with three columns: 'Method', 'Resource path', and 'Integration target'. The first row shows 'GET' in the Method column, '/hello' in the Resource path column, and 'pdcLambda' in the Integration target column. There is an 'Add route' button below the table and a 'Remove' button next to the integration target. At the bottom right, there are buttons for 'Cancel', 'Review and create', 'Previous', and 'Next'.

Method	Resource path	Integration target
GET	/hello	pdcLambda

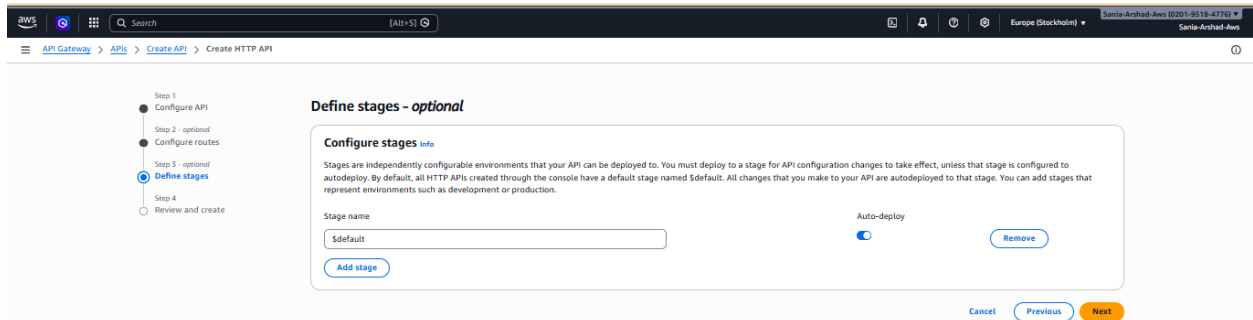
Activate Windows
Go to Settings to activate Windows.

Step 6: Deployment

API deployed using:

\$default stage

Parallel & Distributed Computing



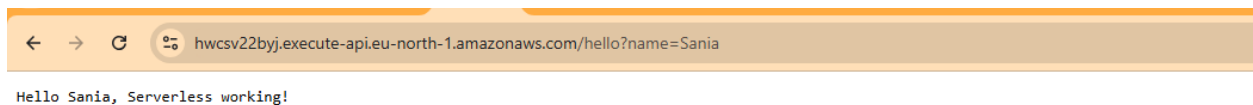
Step 7: Browser Testing

Tested endpoint:

<https://hwcsv22byj.execute-api.eu-north-1.amazonaws.com/hello?name=Sania>

Output:

Hello Sania, Serverless working!



8. Algorithm

Algorithm:

- Receive HTTP request
- API Gateway triggers Lambda
- Extract query parameter
- Process request

- Return response

9. Results

The serverless application successfully generated dynamic output using AWS Lambda and API Gateway.

Final output:

Hello Sania, Serverless working!

The project successfully validated serverless execution.

10. Advantages

Advantages of the system:

- No server management
- Automatic scaling
- Cost effective
- High availability
- Faster deployment
- Event-driven processing

11. Challenges Faced

During implementation the following issues were encountered:

- AWS billing verification errors
- API route integration problems
- Deployment troubleshooting
- Endpoint configuration issues

These were resolved through proper Lambda integration and API deployment.

12. Applications

Applications of serverless architecture:

- Web APIs
- Chatbots

- Automation scripts
- Data processing
- IoT systems
- Event driven applications

13. Limitations

Some limitations include:

- Cold start delays
- Timeout limits
- Vendor lock-in
- Debugging complexity

14. Future Enhancements

Future improvements can include:

- Database integration
- Authentication
- File processing
- PDF generation
- Full serverless web application

15. Conclusion

This project successfully implemented a serverless application using AWS Lambda and API Gateway under AWS Free Tier.

- The system demonstrated:
- Event driven execution
- Cloud deployment
- Scalable architecture
- Low-cost serverless computing

The project provided practical understanding of modern cloud-native application development.

16. References

1. AWS Lambda Documentation
2. AWS API Gateway Documentation

3. Amazon Web Services Free Tier Documentation